

RECIPROCAL APPLICATION FEDERATION

Independent apps. Integral capabilities. One coherent experience from either entry point.

Phenotype Atlas & Assurance • Amendment 1.3
September 16, 2026

DESIGN STATUS: PROPOSED

This reading edition formalizes co-install discovery, native host-relative composition, scoped consent, optional App Center, recursive applets and reversible lifecycle. It is not a current source audit, an implemented runtime, an approved migration, or product qualification.

Use the accompanying Markdown, requirements, work graph, examples and per-repository owner packets for execution. One existing owner per repository and prior safety/assurance contracts remain in force.

The governing distinction

Opening a different app changes the perspective—not the data owner, the permission boundary, or the identity of the work.

Sufficiency audit of the attached v1.2 package

Verdict: the conceptual foundation was present; the requested product experience was underspecified. This is an artifact review of the mounted v1.2 ZIP, not a claim about unseen current source implementations.

The existing architecture/APPLET-CONTRACT.md defines recursive applets, provided/required capabilities, domain-facing ports, state and permission ownership, lifecycle/health, locality-aware lowering, child readiness, quiescence and a basic composition test. Its schema includes ports, surfaces and children, but represents federation_policy as an unconstrained string. The example is a synthetic CLI/library component. HLD/ALD/LLD primarily specify the atlas and assurance program, not a native composite-application experience.

Concern	Existing coverage	Amendment
Recursive components and composites	Explicit	Preserve; add bounded re-export and perspective rules
Ports, capabilities, owned state	Explicit high-level contract	Add scoped offers, requirements, recipes and selected bindings
Same-process/IPC/network choice	Explicit high-level guidance	Retain optimized lowering; never make network mandatory
Co-install detection	No complete install/discovery protocol	Signed/custodied manifest discovery with lazy activation
Automatic integration	Not an operational contract	Deterministic, consent-aware recipe reconciliation
Opening either app as primary	Not specified	Host-relative perspective without authority switching
Native command/navigation experience	Generic surface labels	Slot, command, focus, accessibility, context and error contracts
App Center	Not defined	Optional management/workspace view over the same bindings
Multiple simultaneous hosts	Not defined	Shared scoped service leases, independent views, no duplicate mutations
Learned integration	Not defined	Learn preferences; generate proposals, not unreviewed executable bindings
Update/uninstall/data retention	Quiescence mentioned	Versioned plans, leases, recovery journal and owner-safe detachment
Composite scaling	Recursive idea	Lazy UI/service activation, expansion budgets, event-loop control
Cross-app qualification	Basic compose/remove test	Real installation-order, host-swap, failure, privacy and upgrade matrix

A subfolder or trait named 'applet' is not proof of the stronger experience. Conversely, no new universal framework is justified merely because this document identifies missing behavior. Reuse

existing qualified applet/service/UI facilities and add the narrow common contracts the first real consumer pair needs.

The previous schema remains readable for old examples. The new illustrative profile is additive and opt-in; it is not a silent interpretation of every old free-text policy. Production compatibility adapters must explicitly map old records or reject an unsupported profile.

The user's Apple/Google/Microsoft/Linux Foundation/GNU/Apache references are interpreted as desired qualities: immediate coherence, reliable lifecycle and recovery, platform fit, openness, inspectability, interoperability and durable ownership. They are not assertions that any vendor uniformly satisfies all those properties, nor a requirement to combine their entire stacks.

Product contract: independently useful, better together

Definition

Reciprocal, host-relative application federation is a capability- and policy-driven composition system in which installed applications can provide selected services and surfaces to one another. Each application remains independently usable under its declared base dependencies. The application the user deliberately opens supplies the primary workspace and interaction grammar; other compatible applications contribute contextual capabilities that feel integral rather than like nested full applications.

Reciprocal means that both directions are possible where useful and implemented. It does not require identical features in both directions, every pair to integrate, or a graphics library to become a GUI host. A composition may itself expose a bounded applet contract to a larger composition. A repo, installed package, process, applet, product and window are distinct identities.

The central distinction

Perspective changes; authoritative ownership does not. Opening Tracera can foreground a product graph and show an AgilePlus work panel for the selected capability. Opening AgilePlus can foreground the work item and show Tracera's product context and dissatisfaction. The same work item and product entity remain the same records. Neither program manufactures a second database to make its own UI look self-contained.

Primary is relative to a window/workspace/session, not a global 'last launched app wins' setting. Two windows can simultaneously have different primary apps without oscillating defaults, stealing focus or changing the user's chosen provider. A background launch cannot reorder visible navigation or take over the primary perspective. A user can explicitly reframe a workspace while retaining its stable object context.

Five operations that must never be conflated

1. **Source consolidation:** development-time changes to repositories and package ownership, with separate authorization.
2. **Installation:** place a signed/qualified package in an allowed environment and register its capabilities.
3. **Runtime composition:** select and connect compatible providers to consumers under a scoped plan.

4. **Experience composition:** present contextual commands, views, objects and workflows coherently in a host.

5. **Data migration:** change data format/ownership/storage under explicit migration rules; not an automatic side effect of composition.

The user's 'absorbed' feeling is chiefly operations 3 and 4. It does not mean the other app vanished or its state was annexed.

Discovery and automatic setup

When B is installed after A, eligible hosts observe metadata changes, validate artifact identity and the declared compatibility profile, and reconcile approved recipes. Safe integration inside an already-approved principal/workspace/resource scope can activate without repeated dialogs. New sensitive scopes, new external transfers, purchases, installs, destructive commands, background services or ambiguous providers require the appropriate explicit decision.

The reverse install order yields the same usable capability set under the same selected versions/policy. Discovery alone does not start B's complete desktop UI or its expensive backend. No manual path/port/API-key copying is required for the qualified first-party local integration path. Credentials are not copied between apps: use scoped brokers/references and independent enforcement by the authoritative provider.

A missing optional peer leaves a useful standalone app, not a broken shell or a wall of advertisements. A missing base dependency is reported honestly. Installing C must not silently replace a user-pinned B provider or add every C command to every menu.

App Center

The optional App Center provides discover/install/update/repair, capability and permission inspection, provider choices, workspace presets, conflict explanation, detach/export and recovery. Its user-visible model is 'apps and useful capabilities', not a compulsory giant dashboard.

It is another perspective and consumer of the same APIs, not a second installation database, work ledger or source authority. Apps can activate integrations and work offline under cached approved local plans without opening the Center. An always-on cloud account is not required for qualified local composition. Cloud or multi-device dependencies remain explicit where the workflow needs them.

Packaging can begin as a view or applet in an existing accepted shell (for example a scoped PhenoApps consumer), not an automatically approved new repo/product. The correct home follows actual consumer and release ownership, not the aesthetic appeal of a platform name.

'Learning to merge'

Use deterministic discovery, typed compatibility and explicit recipes before learning. The system may learn that the user prefers B for previews, retains a certain panel layout or repeatedly creates a useful composition. The result is an inspectable preference or suggested recipe, scoped to its environment and easy to reset.

A model may propose an adapter as development work. It cannot invent compatible contracts, execute generated glue on installation, infer permission from co-presence, migrate data, accept itself as verified, or change a user's default invisibly. Confidence is not authority. Declining a suggestion should be durable and quiet.

Proposed first product witness

Use Tracera and AgilePlus after confirming both standalone paths. Bind one real subject, capability and work item; display actual work inside Tracera, then actual product context inside AgilePlus. Keep each data authority intact and obtain real work-engine receipts. Update an item, open both perspectives, restart one provider, revoke a write scope, detach and reinstall. Verify continuity and audit the exact native UI/keyboard/accessibility paths. This is planned, not an assertion that either host already implements it.

A second distinct pair (for example BytePort + ShareCLI) tests whether the contracts are truly reusable. A third nesting test exposes a composite's reduced public capability without recursively embedding complete app chrome. Broader automatic composition comes after these witnesses, not before useful product delivery.

Seamless experience means shared interaction, not shared wallpaper

One host grammar

The host owns its navigation hierarchy, toolbar placement, command palette, focus policy, accessible structure and primary task vocabulary. An extension supplies bounded contributions, not a second menu bar and sidebar wrapped in another window. The same task remains recognizably the same task in either perspective; words can be contextualized without changing its semantics.

A contribution declares a stable command/view ID, target object kinds, accepted slot types, context predicate, enablement predicate, help, permission need and unavailable/error behavior. Context predicates control discoverability, not authorization. The provider rechecks permission and preconditions when the command executes.

Slots include detail inspector, contextual panel, primary task view, inline editor, preview, result card, command, context menu, search provider, notification and headless operation. A host advertises supported versions, allowed isolation modes, layout constraints and lifecycle. Unknown slots do not become arbitrary HTML injection points.

Shared context, distinct authority

Pass explicit object references, schema/version, workspace/principal, selection, permitted actions and bounded navigation state. Avoid global mutable 'current project' variables, filesystem-path identity, and implicit sharing of every selected object with every app. Different Windows/macOS/Linux sessions may map to the same person only through an explicit trusted identity relation, not matching usernames.

Deep links resolve stable objects, then choose a compatible installed host under user preference. A link may open a work item in AgilePlus or its integral surface in Tracera without duplicating the underlying record. Deleted, forbidden, migrated and incompatible targets have distinct explanations. Back/forward history does not bounce between top-level application launchers.

Focus and command routing

Exactly one focus owner handles an input event in a view. Resolve shortcut conflicts through explicit host precedence and user overrides; retain accessibility and platform shortcuts. Relative mouse capture, IME, pen, drag/drop and clipboard require declared support for specialist surfaces. A remote/pixel surface cannot claim native accessibility or text semantics it does not expose.

Opening another app does not steal an existing editing session or cancel its work. Commands carry stable operation IDs and expected versions. UI retries cannot repeat non-idempotent actions with a new operation ID. A global command palette does not imply a global undo stack: commands advertise local undo, compensating action or irreversibility, and the UI explains partial multi-owner outcomes.

Settings, search and notifications

Provide one contextual settings entry with namespaced ownership, precedence and effective-value explanation. App-local settings do not silently become workspace-wide settings. A shared search view performs permission-filtered queries and deduplicates by authoritative object ID; it does not merge all databases or leak private result counts/snippets. Search/backend failure remains visible without blanketing the whole host.

Deduplicate notifications by operation and audience. Show a single status/progress surface for shared work, with authoritative owner available in details. Avoid duplicate trays, sign-in dialogs and progress bars. Errors should identify the failed capability and safe recovery action without requiring the user to understand repository boundaries.

Native quality and visual restraint

Preserve platform-appropriate keyboard behavior, menus, drag/drop, touch/pen, high-DPI scaling, screen-reader semantics and OS-level lifecycle. Reuse the accepted design tokens and editable asset lineage. Theme conformance does not justify hiding provenance or impersonating another publisher. Integrated areas may expose unobtrusive provider details in an inspector, permissions page or troubleshooting view while remaining part of the primary experience.

The desired richness is purposeful: layered/2.5D/3D assets, smooth transition, good materials and contextual motion where they clarify an action. Supply reduced-motion, reduced-transparency, high-contrast and static/low-power fallbacks. Do not place an always-running 3D canvas behind routine tables. GPU/CPU/audio budgets and cold/warm paths must be profiled with actual foreground contention.

Screenshots are evidence of pixels, not proof of interaction. Native accessibility checks, keyboard journeys, wrong-context commands, multiwindow behavior and failure recovery must accompany the visual review. No universal millisecond budget is invented here: each selected profile declares and measures its own frame/input/startup/RAM/energy limits relative to standalone baselines.

Standalone, embedded and app-center modes

The same capability may have a full standalone experience and several embedded views. Share domain logic and use host adapters; do not fork the feature implementation per host. A full standalone shell is not the default embedded contribution. A provider can offer a compact inspector without exposing all its settings and unrelated features.

CLI-only tools contribute actual typed operations, job status, results and optional inspectors. A host can render those surfaces without turning the tool into an invented GUI product. CLI stdout remains a machine contract: no extra banners or UI-control escapes in JSON/MCP output. VHS captures

actual standalone and host-mediated command journeys.

Concrete 'feels like one app' acceptance

Select an object in A; invoke a contributed B action without copying IDs, ports or credentials; see appropriate progress and errors; edit through the real B authority; undo or compensate where declared; search for the object once; open the same object from B with equivalent state; use keyboard and assistive technology; restart and recover. At no stage should the user need to understand a repository boundary just to complete the qualified workflow.

Runtime architecture and explicit contracts

Distinct planes

Plane	Responsibility	Authority boundary
Package/discovery adapters	Observe approved installed artifacts and capability manifests	Do not execute arbitrary package hooks to discover metadata
Resolver and composition planner	Match requirements, versions, trust, scopes, locality and resources	Produces a proposed plan, not new permission
Policy and grants	Decide whether a binding may activate and execute	Provider still enforces scope on every operation
Service activation	Start or lease the needed provider instance lazily	Avoid full application startup for a single capability
Experience host adapters	Mount views and route commands/context coherently	Host chrome and focus are not data ownership
State/data adapters	Resolve object handles, versions and transformations	Exactly one declared writer authority per entity/field contract
Diagnostics	Explain selected providers, failures and drift	Observation does not authorize mutation

Reuse qualified native extension catalogs, IPC, registries and parsers. A shared library can implement neutral contracts; installations hold their own scoped runtime binding state. This must not become hardcoded portfolio data inside AgilePlus or PhenoDocs. An installation/runtime capability catalog is useful infrastructure, not the portfolio's product-truth registry.

Inputs and negotiated output

An installation manifest identifies product/package/artifact version, publisher, allowed activation endpoints, provided and required capabilities, supported host surfaces, platform profile, owned state, lifecycle, resource constraints and extension recipes. A recipe is declarative and versioned, naming match conditions, context mapping, contributions, permission requirements and failure/detach policy. It is not arbitrary install-time code.

A composition plan pins package/contract versions and artifact identities; binds consumer capabilities to providers; records principal/workspace scope, approved grants, transport/isolation, service instance keys, surfaces, startup dependencies, rollout/recovery and expiration/revalidation conditions. Synthetic examples in this amendment illustrate shape only. A signed plan or matching schema cannot prove behavioral compatibility by itself.

Compatibility has at least five parts: type/interface compatibility, semantic behavior, lifecycle/error behavior, security/resource requirements and host-surface support. A matching string such as tasks is insufficient. Probe declared handshake/conformance fixtures before activation. Missing required capabilities yield a concrete unresolved plan, not a null provider or successful empty output.

Resolution order

Filter by visibility/trust/platform first, then by version and semantic contract, then by required scopes and resources. Prefer the user's valid pinned provider. Otherwise use approved workspace policy and tested recipes, with deterministic tie-breaking and explicit ambiguity where choices matter. Do not equate newest, local, cheapest, largest or first-discovered with always best.

Do not recompute all pairwise integrations continuously. Index typed capability offers; maintain affected bindings incrementally; retain known-good plans when inputs are unchanged; invalidate only dependent plans when packages, policy or contracts change. Explicitly report incomplete observations. Optional capability discovery must not materialize every UI view or load every implementation.

Host-relative composition

A perspective record identifies the host application/window, root context, selected recipe set, view layout and user customizations. A binding record identifies the provider, consumer, contract and execution scope. A service lease identifies the actual instance and references. These are different records so opening a second perspective does not automatically duplicate a job or shift the data authority.

Two hosts can use one provider instance only where identity, configuration, trust, isolation, resource and tenancy scopes agree. Identical command text does not make computations safely shareable. A conflicting scope creates separate instances or fails according to policy, never leaks data to save memory.

Recursion and cycles

Composites re-export only an intentional bounded contract, including required dependencies and failure behavior. A product-level applet need not reveal internal provider names. A composite can be embedded by another host while retaining the underlying child provenance.

Visual mounting and strict startup prerequisites must be acyclic in the initial implementation. Cross-calling workflows may contain cycles only through explicit asynchronous/event protocols with origin, correlation, deduplication and hop/rate bounds. Reciprocal perspectives are not reciprocal full-shell embedding. Stable instance IDs and reentrancy rules stop an A panel inside B from recursively mounting all of A's B panels.

Locality and fast paths

Do not convert every boundary to RPC. Same-process qualified components can use direct calls; same-OS processes can use appropriate IPC/shared-memory/handles; VM paths can use qualified shared-memory/device interfaces; LAN/WAN paths use negotiated transport and serialization. Preserve existing PhenoFabric locality and deadline goals without pretending arbitrary code or views can be remotely placed.

Security boundaries justify unavoidable copies and checks. Zero-copy is a measured implementation property of a specific route, not a promise inferred from a shared-memory field. Heavy rendering/audio/GPU providers activate only for demanded work and remain under resource

admission; background discovery is event-driven and bounded.

Shared failure domains

A minimal local broker is a possible implementation choice, not an excuse for a single fragile mandatory daemon. Define what survives broker restart: existing safe connections, known-good bindings, no new grants, and reconnect identity. No unbounded auto-restart storm. Expose a disabled-integration mode so the host's independent functions remain usable while debugging a provider.

Installation, activation, upgrade and removal

State model

Track installation status separately from binding status and instance status. A package can be installed while no integration is allowed or running. A failed binding does not prove the package is absent.

Binding states include DISCOVERED, INCOMPATIBLE, FORBIDDEN, AWAITING_CONSENT, RESOLVED, PREPARING, ACTIVE, DEGRADED, QUIESCING, DETACHED and FAILED_RECOVERABLE. Every transition records scope, plan revision, cause and actual operation outcome. These are proposed state names, not fabricated deployed APIs.

Co-install reconciliation

1. Observe a verified package event or bounded inventory change in the correct principal/environment.
2. Read manifests without executing providers. Distinguish partial inventory from an empty installation set.
3. Resolve qualified recipes and required capabilities using pinned policy and current grants.
4. Present one meaningful approval when needed; auto-activate only inside a previously authorized envelope.
5. Prepare scoped service endpoints and UI contributions without exposing a half-initialized feature as ready.
6. Commit a locally durable plan and activate ready contributions. Mark required versus optional failures correctly.
7. Retain a recovery record and explicit detach operation. Probe readiness instead of inferring it from process existence.

Package watchers are at-least-once observations, not a promise of exactly-once callbacks. Stable event/operation IDs, expected versions and idempotent reconcile operations prevent duplicate services/menus/jobs. Repeating an unknown external action with a new ID is prohibited until its result is reconciled.

Transaction limits

The planner may transactionally update its own state. It cannot honestly promise a global ACID transaction over unrelated apps, stores, package managers and remote services. Prepare/commit where providers support it; otherwise use a recoverable sequence with explicit compensation and partial-state reporting. Keep domain data migration outside implicit co-install setup.

If the process dies after a provider accepted an action but before the host saw its receipt, recover by operation ID. A compensating action may not erase an irreversible external result; surface that limitation. Do not label all rollback scripts successful merely because they exited zero.

Hot update and version skew

Pin the active composition. Validate candidate versions and capabilities without swapping underneath an in-flight edit or live audio/game path. Quiesce at a safe boundary; drain or explicitly cancel work; checkpoint owned state; atomically swap local routing where supported; then verify readiness and retain rollback artifacts.

True hot unload is optional and must be advertised. Some native libraries, processes or plugin environments require a restart. Display that requirement rather than freeing an in-use pointer, hanging the host or promising no interruption. Allow side-by-side provider versions when their scope and state formats permit it. A new schema version needs an explicit compatibility/migration plan, not blind downgrade on rollback.

Detach versus uninstall

Detaching B from A removes that binding and its UI contribution, revokes the grants specific to that relation, releases leases and preserves B's data. It does not uninstall B or affect its unrelated consumers.

Uninstalling B must identify dependent bindings and distinguish base-required from optional capabilities. Required workflows become blocked with a repair route; optional enhancements disappear gracefully. Stop instances only when no valid leases remain. A package manager's removal hook cannot be assumed to run reliably; stale registration cleanup must be idempotent and recoverable.

Data deletion is a separate explicit decision with export/retention rules. The federation system may remove its own stale cache and grants, not delete the provider's authoritative user data. Reinstallation under a new principal must not inherit another person's old access grants. Same publisher name is not sufficient to trust a replacement artifact.

Multi-window and background operation

A user can close a host window without terminating a provider job that has an accepted background policy. Closing the last UI lease is not always cancelling the job lease. Conversely, providers cannot remain indefinitely active without a permitted owner and budget.

Startup reconstructs the current perspective and resolves still-valid object references. It does not blindly replay UI actions. Launching both A and B must not start duplicate migrations, schedulers, capture services or notifications. Inactive user sessions and remote nodes advertise availability honestly.

Recovery evidence

For each stage, test abrupt termination, repeated events, stale plans, permission revocation, provider crash, dependency disappearance, update skew, disk-full conditions and lost acknowledgments. Attach actual source/package/profile identities and logs. Success means the world is in a known recoverable state and no protected data/action was duplicated—not that every error was suppressed.

Security, consent and data authority

Co-installation is not consent

Two applications being installed by the same OS user does not imply permission to share files, secrets, workspaces, identities, microphone/camera, screen capture, remote devices or external accounts. A publisher signature proves an origin under a trust policy, not harmless behavior or blanket access.

Safe automatic setup requires a validated recipe, compatible interfaces, a trusted artifact/profile, explicit principal/workspace binding and grants sufficient for that particular operation. New privilege scopes do not inherit approval from an older recipe version. The runtime policy and each provider must both enforce access; hiding a button is not authorization.

Offer policy modes such as manual-only, suggest-integrations and auto-enable-qualified-low-risk-integrations. Do not silently select a high-automation mode. A deployment can preauthorize scoped first-party recipes with clear provenance and revocation. The user should not need to repeatedly approve the same unchanged safe binding, but a cross-boundary change needs reconsideration.

No universal shared credential bag

Use native secure storage and suitable established authentication/brokering facilities. Bind delegated authority to audience, principal, workspace/resource, permitted action, expiry and revocation. Pass references or scoped tokens rather than copying a provider's API key into another app's configuration. Do not serialize real credentials in manifests, prompts, traces, screenshots, or package examples.

An agent's broad local filesystem access is not the correct model for installed product integration. Sandbox boundaries and platform permission dialogs remain real constraints. Brokered helpers must avoid confused-deputy behavior by verifying the calling identity and the exact requested resource.

Authority map

Define owned entities/fields, write APIs, accepted versions, consistency, caching and transformations. A host may keep a bounded projection/cache with provenance and freshness; it does not become a second source authority. Simultaneous hosts use expected revisions/conflict semantics rather than last-writer-wins by accident. Cross-app undo requires explicit command semantics and compensation.

One sign-in experience is possible only through a qualified identity/broker contract; it does not automatically unify unrelated tenant accounts or paid entitlements. An external permission or entitlement outage cannot be hidden by borrowing another app's cached token. Keep security-sensitive prompts native and distinguish provider-owned content from trusted host

chrome.

Isolation choices

In-process native code is a high-trust execution mode. Matching an interface or loading through Module Federation does not sandbox it. A webview requires restricted bridge APIs, origin/CSP policy and validated messages; an iframe and a remote module are not equivalent security boundaries. Out-of-process and Wasm adapters have their own host-call and resource limits. A container alone does not prove all requested isolation.

Validate manifest size, URLs/paths, allowed activation methods, input schemas, rendering payloads and artifact provenance. Reject path traversal, symlink escapes where material, unsafe URI schemes, origin confusion and unexpected code downloads. A supplied integration manifest is data; text within it cannot override policy or act as agent instructions.

Privacy-preserving discovery and adaptation

Local discovery should reveal only the metadata needed for eligible integration. Background analytics and cross-device advertisements require their own scope. Do not broadcast installed applications or private workspace names to unrelated peers. Learning stays bounded to approved preferences and can be inspected/reset; raw application content is not training input by default.

Search results, notifications, cached views, embeddings and captures inherit source sensitivity. Denied objects must not leak through titles, counts or stale previews. Revocation updates UI and outstanding operation handling as well as future API requests; it must not fabricate completion of already-executed effects.

Exportability and open integration

Users can inspect selected providers, permissions, versions, reasons and state locations. Offer documented export and detach, not a proprietary universal store that makes leaving impossible. Third-party implementations can conform to published contracts, run shared fixtures and join under the same policy. No implication of certification by Apple, Microsoft, Google, GNU, Apache or a foundation is made by following these principles.

Reuse-first platform lowering

The semantic contract is cross-platform; the execution and visual adapters are platform-specific where needed. Keep the matrix current with actual OS/SDK versions and packaging/entitlement constraints. These researched sources establish available mechanisms, not that the portfolio has already integrated them. See sources.

Mechanism	Useful part	Important limit / proposed role
Apple ExtensionFoundation + ExtensionKit	Declared extension points, discovery, XPC, optional extension UI and sandbox policy	Version/entitlement-specific; not universal embedding of any arbitrary installed app. Prototype native host/extension behavior on the supported Macs.
Windows app extensions + app services	Package-catalog discovery, lifecycle events, two-way service integration	Documented app-extension path requires package identity. Keep unpackaged behavior explicit.

Mechanism	Useful part	Important limit / proposed role
OLE in-place activation	Established host/container presentation with contextual menus and embedded objects	Windows-specific historical mechanism; valuable UX precedent, not universal modern core.
XDG Desktop Portals + D-Bus	User-mediated scoped desktop resources, document handles, service activation	Portal availability depends on environment/backend. Portals are not an arbitrary UI compositor.
OSGi capability resolver / Apache Felix	Explicit requirements, providers, transitive resolution, dynamic service lifecycle	Excellent reference/reuse candidate where its runtime fits; do not add a JVM to every Rust app without need.
VS Code / Eclipse Theia	Commands, context predicates, views, service and host extension models	Existing domains have specific APIs and lifecycle. Theia distinguishes compile-time extensions from runtime plugins; do not pretend all are interchangeable.
Module Federation	Dynamic remote module loading and dependency coordination in web hosts	Not security, permission, lifecycle or domain-semantic proof. Use only with matching web runtime/trust profile.
WIT / WebAssembly Component Model	Typed import/export boundaries and composition of components into components	Interfaces do not specify behavior; components do not offer arbitrary cross-component shared-memory semantics. Select for suitable portable isolated computation.
Native direct calls / qualified ABI	Lowest-overhead local composition	ABI/version/ownership and trust must be explicit. No forced network round trip for same-process work.
Window/video remoting	Compatibility fallback for applications with no semantic integration	A picture is not a native semantic applet. Declare degraded accessibility, focus, clipboard, DPI/HDR and latency behavior.

Surface strategy

Prefer a native or host-native contribution adapter using a shared domain API. For simple inspectors/forms, evaluate a constrained declarative view model rendered natively by each host. For complex surfaces, use a dedicated qualified native adapter or isolated webview/canvas with the required input/accessibility bridge. Do not handroll an entire universal UI toolkit merely to make every app look similar.

Maintain explicit conformance tiers: linked launch, contextual handoff, embedded surface, native semantic contribution, and full workflow federation. A lower tier can be useful. It cannot claim a higher tier because its panel shares the color palette.

Locality and rendering

A Swift UI subtree cannot be assumed embeddable inside arbitrary Tauri/GTK/WinUI/Bevy windows with no adapter. An installed app requires exposed contracts or a maintained integration adapter. An engine/rendering surface may need its own event loop, device/context, process and output transport. The runtime negotiates a valid path or advertises a lower integration tier.

Use PhenoFabric's existing locality work when a surface/service actually crosses processes, VMs or devices. The federation layer chooses the capability and perspective; transport should not decide who owns product data. Shared GPU resources, audio clocks, input capture and color conversion retain their technical qualification requirements. No universal AV1 or zero-copy assumption belongs

in the federation contract.

Build and distribution

Standalone and integrated modes should share authoritative implementation packages and tests. Version the contribution SDK and integration recipes independently where useful, while recording tested combinations. Reuse supported package/discovery systems and existing app release pipelines. Avoid a central deployment that replaces all applications merely to enable one integration.

Every installed host/provider combination needs real package installation, process identity, permission and update testing—not only source-tree module tests. Keep upstream licenses, attribution, maintained patches, entitlement requirements and an exit path visible. Reuse our own capabilities as upstreams before copying them per host; choose an external facility where it better satisfies the contracts.

Research and comparative proof

Claim being tested

Hypothesis: reciprocal semantic composition reduces setup/context-switching/maintenance burden while meeting the standalone products' critical correctness, privacy, performance and accessibility obligations. The mechanism is typed capability reuse plus host-relative surfaces and scoped state—not a new launcher or visual skin.

Competing hypotheses: contextual deep links are sufficient; a conventional plugin host is simpler; a pre-integrated suite is better; pair-specific adapters are cheaper for the small real integration graph; federation introduces too much versioning and security complexity. Test these before claiming an ecosystem moat.

Baselines

For the same user job, compare (A) separate qualified apps plus documented manual integration, (B) deep-link/context handoff, (C) one conventional plugin-host composition, (D) proposed reciprocal federation, and where relevant (E) an established integrated suite. Use primary-source mechanisms in SOURCES as implementation references, not as fictitious drop-in competing products.

This amendment is not another 25-product market survey. Existing per-product SOTA longlists remain in force. Here the question is whether this particular interaction/composition mechanism improves their real common workflows. Do not pad the reference list with unrelated projects to reach a count.

First witness: Tracera + AgilePlus

Each app must work independently on the selected supported profile. Bind one real product/capability/work item. Run install A then B and B then A; open each as primary; execute a real scoped edit with an actual receipt; navigate to the same object from both; open both at once; terminate and restart a provider; revoke write access; update one app; detach/uninstall B; prove A's base workflow and both apps' retained data.

Use a nontrivial seeded project with known missing requirements, conflicting changes and private data that another context must not see. No synthetic dashboard, placeholder backend or manual

database copy can establish reciprocal federation.

Second and nested witnesses

BytePort + ShareCLI is a suggested distinct pair: deploy/operate from BytePort while ShareCLI offers scoped process/control surfaces, then inspect the same authorized workload from ShareCLI while BytePort provides deployment context. Qualification must retain ownership and dangerous-action boundaries; two UIs cannot initiate duplicate retries or unapproved rollbacks.

For a nested composition, host a reduced composite inside an existing approved shell or App Center view. Check that it exposes only intentional capabilities, not all child secrets/settings. Stop recursive UI embedding and event loops. Choose actual runnable pairs; these suggestions do not reassign a repository or authorize source movement.

Measurements

Predeclare exact hardware/OS/builds, isolated environments, task corpus, user expertise, permissions and failure schedule. Measure setup actions and failure rate, manual identity/path/credential copying, context-switch count, task completion/error rate, provider diagnostic quality, install/update/detach time, retained-data integrity, command latency distribution, startup, memory, GPU/CPU use and load interference.

Assess developer ergonomics too: effort to add a provider and a second host, host-specific code versus reused domain logic, change amplification, number of pair-specific adapters, version compatibility burden, documentation ambiguity, debugging time and external-author onboarding. Count patches and duplicated policy rather than celebrating framework LOC.

Visual/interaction review should use qualified native output and assistive/keyboard journeys. Blind task-based review where practical is preferable to awarding a perfect score because the author chose attractive screenshots. Retain uncertainty, failed runs and exact integration-tier limitations. Benchmarks are unexecuted until native receipts exist.

Required adversarial matrix

Include no-peer, unsupported-peer, ambiguous-provider, mismatched-version, untrusted-manifest, revoked-scope, two tenants, slow/crashing provider, no network, partial install, unknown previous operation outcome, concurrent hosts, hot update with in-flight action, broker restart, uninstall with another consumer, URI injection, context substitution, and event-loop storms. Use pairwise configurations for routine breadth but fully test critical boundaries. Pairwise coverage is not proof of every higher-order interaction.

The previous independent 85%+ suite obligations and stronger accepted floors remain. Critical federation invariants require complete evidence; aggregate standalone coverage cannot hide untested composed paths. A schema pass is not permission, an attestation is not correctness, and a screenshot is not a successful interaction.

Graduation

Graduate a binding profile only after standalone parity, both declared directions, the lifecycle/failure matrix and the user's native experience review succeed. Record any disadvantage and its audience. If the mechanism is not beneficial, retain a simpler deep-link or conventional plugin solution and preserve the unfulfilled long-term intent as research. Do not market an automatic union of arbitrary third-party applications.

Execution and evidence index

Acceptance model

56 proposed federation requirements and 56 planned acceptance scenarios are in `federation/REQUIREMENTS.md` and `ACCEPTANCE.md`. They cover identity, discovery, experience, lifecycle, security, composition and assurance. Product tests and pilots remain unexecuted.

Existing owner packets

All 24 folders in the unchanged v1.2 snapshot have `FEDERATION.md`. Publication/library/game/CLI owners receive appropriate roles, not an instruction to build a universal GUI. The shared owner matrix is `federation/OWNER-MATRIX.md`.

Integration plan

12 work packages extend the existing work DAG. The first pair is a proposed Tracera-AgilePlus witness; a second distinct pair and nested composite guard against overfitting. Timings remain unestimated until actual code/native profiles are bound.

Reference schemas

Illustrative manifests and two reversed perspectives are synthetic. Offline checks exercise schema, identity, lock, scope, authority and cycle consistency. They do not authenticate permissions or certify native behavior.

Source reuse

The following primary documentation supports available mechanisms and constraints. It does not imply endorsement, uniform vendor quality, exhaustive research, or completed portfolio integrations.

FED-S01 — [Apple: Adding support for app extensions](#)

Custom extension points, discovery, XPC, scope and UI support through ExtensionKit. Platform/SDK/entitlement constraints apply; not arbitrary app embedding.

FED-S02 — [Apple: App Extensions](#)

Native extension contexts and supported extension surfaces; a limited contract, not whole-app fusion.

FED-S03 — [Microsoft: Create app extensions for Windows App SDK apps](#)

Package-catalog discovery, host/provider roles and install/update lifecycle. This documented path requires MSIX identity.

FED-S04 — [Microsoft: Implementing In-Place Activation](#)

OLE demonstrates embedded object interaction and context-sensitive composite menus. Windows-specific precedent; not the proposed universal runtime.

FED-S05 — [OSGi Core 8: Resolver Service](#)

Requirement/capability wiring, transitive resolution, contexts and diagnostic failures; reuse ideas or implementation where appropriate.

FED-S06 — [Apache Felix Framework](#)

Existing OSGi R8 core implementation. Candidate only where language/runtime/operational costs fit.

FED-S07 — [XDG Desktop Portal: For App Developers](#)

Desktop-resource integration with user-controlled permissions; not general GUI federation.

FED-S08 — [XDG Desktop Portal: Documents](#)

Scoped document access and permission revocation; portal deletion removes an entry, not the underlying file.

FED-S09 — [VS Code: Commands](#)

Command contributions, context conditions, enablement and lazy activation; useful host-adapter reference.

FED-S10 — [Eclipse Theia: Extensions and Plugins](#)

Different compile-time and runtime extension models; headless and frontend lifecycle must not be conflated.

FED-S11 — [Module Federation: Runtime API](#)

Dynamic web remote-module and shared-dependency APIs; not a sandbox or data-authority contract.

FED-S12 — [WebAssembly Component Model: Worlds](#)

Import/export boundaries and host/component roles; type surfaces do not establish behavior.

FED-S13 — [WebAssembly Component Model: Components](#)

Recursive component composition; component memories are not exported/imported as shared memory.

FED-S14 — [Microsoft: Extensibility overview](#)

Combines extension discovery and app services; optional package and service models have different purposes.

FED-S15 — [Eclipse Theia: Composing Applications](#)

Existing configurable application composition and reuse; not necessary to build a new IDE shell for every product.